SENA	**CENTRO DE GESTIÓN Y DESARROLLO SOSTENIBLE SURCOLOMBIANO** **INSTRUCTIVO PARA LA EJECUCIÓN DE LA FPI**

Programa de Formación: Análisis y Desarrollo de Software.

ID ficha: 3312744

Competencia: Desarrollar la solución de software de acuerdo con el diseño y metodologías de desarrollo.

Resultado de Aprendizaje: Construir la base de datos para el software a partir del modelo de datos.

Actividad de Aprendizaje: Actividad de transferencia del conocimiento — Desarrollar la Api Rest con Python del proyecto formativo de acuerdo con los requerimientos suministrados por la unidad productiva.

Instructivo: EA4 — Construir el prototipo de una Api Rest con Python que permita hacer un CRUD a la base de datos del proyecto formativo de acuerdo con los requerimientos suministrados por la Unidad Productiva de Gastronomía.

Unidad Productiva: Gastronomía — Centro de Gestión y Desarrollo Sostenible Surcolombiano (CGDSS).

(Actividad Individual)

Descripción de la Actividad	**Presentación**: Las API REST permiten que distintas aplicaciones (web, móviles o de escritorio) consulten y actualicen la información almacenada en una base de datos de forma centralizada, segura y estandarizada. En esta actividad de transferencia, el aprendiz aplicará lo aprendido sobre Python, servicios web, bases de datos relacionales (MySQL) y no relacionales (MongoDB) en un contexto real: la Unidad Productiva de Gastronomía del centro de formación. A partir de los requerimientos suministrados por la unidad productiva, el aprendiz construirá el prototipo de una API REST que permita realizar operaciones de creación, consulta, actualización y eliminación (CRUD) sobre la información de productos, recetas, insumos, proveedores, clientes, pedidos y compras, protegiendo cada servicio con autenticación por token (JWT). Como anexo obligatorio, el aprendiz documentará la base de datos del proyecto utilizando el formato Manual de Configuración de Base de Datos.
Listado de recursos, materiales, equipos, herramientas	Guía de aprendizaje R4 – Construir la base de datos Lista de chequeo EA4 Instructivo AA4 y diapositivas AA4 (Api Rest con Python) Formato 6. Manual de Configuración de Base de Datos (anexo) Requerimientos de la Unidad Productiva de Gastronomía Equipo de cómputo con acceso a internet

	Docker Desktop, MySQL 8, MongoDB 7, MySQL Workbench, MongoDB Compass o Studio 3T Python 3.11+, Visual Studio Code, Postman o Thunder Client, Git y GitHub							
Tipo de Evidencia	Desempeño		Conocimiento		Producto	**X**	No produce entrega de evidencia	
Producto y Forma de Entrega	**Caso de estudio: API REST para la Unidad Productiva de Gastronomía** La **Unidad Productiva de Gastronomía** del Centro de Gestión y Desarrollo Sostenible Surcolombiano elabora y comercializa platos, bebidas, productos de panadería y repostería para aprendices, instructores, funcionarios y clientes externos; además atiende pedidos para eventos institucionales. Actualmente la información de recetas, inventario de insumos, compras a proveedores y ventas se registra en cuadernos, hojas de cálculo y mensajes de WhatsApp. Durante el levantamiento de requerimientos, el líder de la unidad productiva identificó las siguientes dificultades: ✓ No existe un control confiable del inventario de insumos (existencias, stock mínimo y fechas de vencimiento), lo que genera faltantes y pérdidas por productos vencidos. ✓ Las recetas (fichas técnicas) no están estandarizadas, por lo que no se conoce con exactitud el costo de producción de cada producto ni su margen de ganancia. ✓ Los pedidos se reciben por diferentes medios y no se puede consultar su estado (pendiente, en preparación, entregado o anulado). ✓ No hay registro centralizado de las compras realizadas a cada proveedor. ✓ No se generan reportes de ventas por periodo ni de los productos más vendidos para la toma de decisiones. ✓ No se conoce la opinión de los clientes sobre los productos y el servicio. ✓ La información es manipulada por diferentes personas sin control de acceso ni roles. Por esta razón, la unidad productiva requiere **centralizar y gestionar su información mediante una API REST** que, en una fase posterior, será consumida por una aplicación web y una aplicación móvil. **Reto del aprendiz**							

Construir el prototipo de una **API REST con Python** que permita gestionar la información de la Unidad Productiva de Gastronomía mediante operaciones CRUD, utilizando **MySQL** como base de datos relacional principal y **MongoDB** para la información no estructurada (calificaciones de los clientes), aplicando seguridad con **JWT** en todos los endpoints y documentando cada uno de los servicios.

Requerimientos funcionales suministrados por la unidad productiva

Cód.	Requerimiento
RF01	Autenticar usuarios del sistema (correo y contraseña) y generar un token JWT con su rol.
RF02	Gestionar usuarios con roles ADMIN, CAJERO y COCINERO.
RF03	Gestionar categorías de productos (platos, bebidas, panadería, repostería, etc.).
RF04	Gestionar productos con su precio de venta y categoría.
RF05	Gestionar la receta (ficha técnica) de cada producto: insumos y cantidades.
RF06	Calcular el costo de producción de un producto a partir de su receta.
RF07	Gestionar insumos y consultar los que están por debajo del stock mínimo o próximos a vencer.
RF08	Gestionar proveedores y registrar compras de insumos (con su detalle).
RF09	Gestionar clientes por tipo: APRENDIZ, INSTRUCTOR, FUNCIONARIO o EXTERNO.
RF10	Registrar pedidos con su detalle y cambiar su estado.
RF11	Generar reporte de ventas por rango de fechas y de productos más vendidos.
RF12	Registrar y consultar calificaciones de los clientes sobre pedidos y productos (MongoDB).

Reglas de negocio

- ✓ **RN01:** El subtotal de cada ítem del pedido se calcula como `cantidad * precio_unitario` y el total del pedido es la suma de sus subtotales; estos valores los calcula la API, no el cliente.
- ✓ **RN02:** Al registrar una compra, se incrementa el `stock_actual` de cada insumo comprado y se actualiza su `costo_unitario`.
- ✓ **RN03:** Cuando un pedido cambia a estado ENTREGADO, se descuentan del inventario los insumos según la receta de cada producto. Si no hay existencias suficientes, la API responde con error 409.
- ✓ **RN04:** No se puede eliminar un producto, insumo, cliente o proveedor que tenga movimientos asociados; en ese caso se cambia su estado a INACTIVO (borrado lógico).
- ✓ **RN05:** Un pedido ENTREGADO o ANULADO no puede modificarse.
- ✓ **RN06:** Las contraseñas se almacenan cifradas (bcrypt) y nunca se devuelven en las respuestas de la API.

Información que deberá gestionar la API

Para dar solución a la problemática, la unidad productiva requiere gestionar inicialmente las siguientes entidades en MySQL (base de datos `bd_gastronomia`). El aprendiz puede agregar campos o entidades si el análisis lo justifica.

Usuarios

Campo	Tipo sugerido	Descripción
`id_usuario`	INT PK AI	Identificador
`identificacion`	VARCHAR(20) UQ	Documento
`nombres / apellidos`	VARCHAR(80)	Nombres y apellidos
`correo`	VARCHAR(120) UQ	Usuario de acceso
`password`	VARCHAR(255)	Contraseña cifrada (bcrypt)
`rol`	ENUM	ADMIN, CAJERO, COCINERO
`estado`	ENUM	ACTIVO / INACTIVO

Categorías

Campo	Tipo sugerido	Descripción
`id_categoria`	INT PK AI	Identificador
`nombre`	VARCHAR(60) UQ	Nombre de la categoría
`descripcion`	VARCHAR(200)	Descripción
`estado`	ENUM	ACTIVO / INACTIVO

Productos

Campo	Tipo sugerido	Descripción
`id_producto`	INT PK AI	Identificador
`codigo`	VARCHAR(20) UQ	Código interno
`nombre`	VARCHAR(100)	Nombre del producto
`descripcion`	TEXT	Descripción
`precio_venta`	DECIMAL(10,2)	Precio de venta (> 0)
`unidad_venta`	VARCHAR(20)	Porción, unidad, libra…
`id_categoria`	INT FK	Categoría del producto
`estado`	ENUM	DISPONIBLE / AGOTADO / INACTIVO

Insumos

Campo	Tipo sugerido	Descripción
`id_insumo`	INT PK AI	Identificador
`nombre`	VARCHAR(100) UQ	Nombre del insumo
`unidad_medida`	ENUM	g, kg, ml, l, lb, unidad
`stock_actual`	DECIMAL(10,2)	Existencias (>= 0)
`stock_minimo`	DECIMAL(10,2)	Nivel mínimo de alerta
`costo_unitario`	DECIMAL(10,2)	Costo por unidad de medida
`fecha_vencimiento`	DATE NULL	Vencimiento del lote actual

estado	ENUM	ACTIVO / INACTIVO

Recetas (ficha técnica)

Campo	Tipo sugerido	Descripción
id_receta	INT PK AI	Identificador
id_producto	INT FK	Producto
id_insumo	INT FK	Insumo utilizado
cantidad	DECIMAL(10,3)	Cantidad por unidad de producto

Proveedores

Campo	Tipo sugerido	Descripción
id_proveedor	INT PK AI	Identificador
nit	VARCHAR(20) UQ	NIT o documento
razon_social	VARCHAR(120)	Nombre o razón social
telefono / correo	VARCHAR	Datos de contacto
direccion	VARCHAR(150)	Dirección
estado	ENUM	ACTIVO / INACTIVO

Compras y detalle_compra

Campo	Tipo sugerido	Descripción
id_compra	INT PK AI	Identificador de la compra
numero_factura	VARCHAR(30)	Factura del proveedor
fecha_compra	DATETIME	Fecha de la compra
id_proveedor / id_usuario	INT FK	Proveedor y usuario que registra
total	DECIMAL(12,2)	Valor total
detalle: id_insumo, cantidad, costo_unitario, subtotal		Ítems de la compra

Clientes

Campo	Tipo sugerido	Descripción
id_cliente	INT PK AI	Identificador
identificacion	VARCHAR(20) UQ	Documento
nombres / apellidos	VARCHAR(80)	Nombres y apellidos
telefono / correo	VARCHAR	Datos de contacto
tipo_cliente	ENUM	APRENDIZ, INSTRUCTOR, FUNCIONARIO, EXTERNO

Pedidos y detalle_pedido

Campo	Tipo sugerido	Descripción
id_pedido	INT PK AI	Identificador
fecha_pedido	DATETIME	Fecha y hora del pedido
id_cliente / id_usuario	INT FK	Cliente y usuario que registra
tipo_pedido	ENUM	MESA, PARA_LLEVAR, EVENTO

estado	ENUM	PENDIENTE, EN_PREPARACION, ENTREGADO, ANULADO
total	DECIMAL(12,2)	Total calculado
detalle: id_producto, cantidad, precio_unitario, subtotal		Ítems del pedido

Colección en MongoDB (base de datos `bd_gastronomia_nosql`): `calificaciones`. Ejemplo de documento:

```
{
  "id_pedido": 125,
  "id_cliente": 18,
  "fecha": "2026-10-20T12:30:00",
  "calificacion_servicio": 5,
  "comentario": "Excelente atención y buen sabor",
  "productos": [
    { "id_producto": 4, "nombre": "Almuerzo ejecutivo",
"calificacion": 5 },
    { "id_producto": 9, "nombre": "Jugo de lulo", "calificacion":
4 }
  ]
}
```

Modelo general de relaciones

Los aprendices deberán analizar las relaciones entre las entidades. Una propuesta inicial sería:

- ✓ Una **categoría** agrupa muchos **productos** (1:N).
- ✓ Un **producto** se elabora con muchos **insumos** y un insumo se usa en muchos productos (N:M), relación resuelta con la tabla **recetas**.
- ✓ Un **cliente** realiza muchos **pedidos** y un **usuario** registra muchos pedidos (1:N).
- ✓ Un **pedido** contiene muchos productos a través de **detalle_pedido** (N:M).
- ✓ Un **proveedor** tiene muchas **compras**, y cada compra contiene muchos insumos a través de **detalle_compra** (N:M).

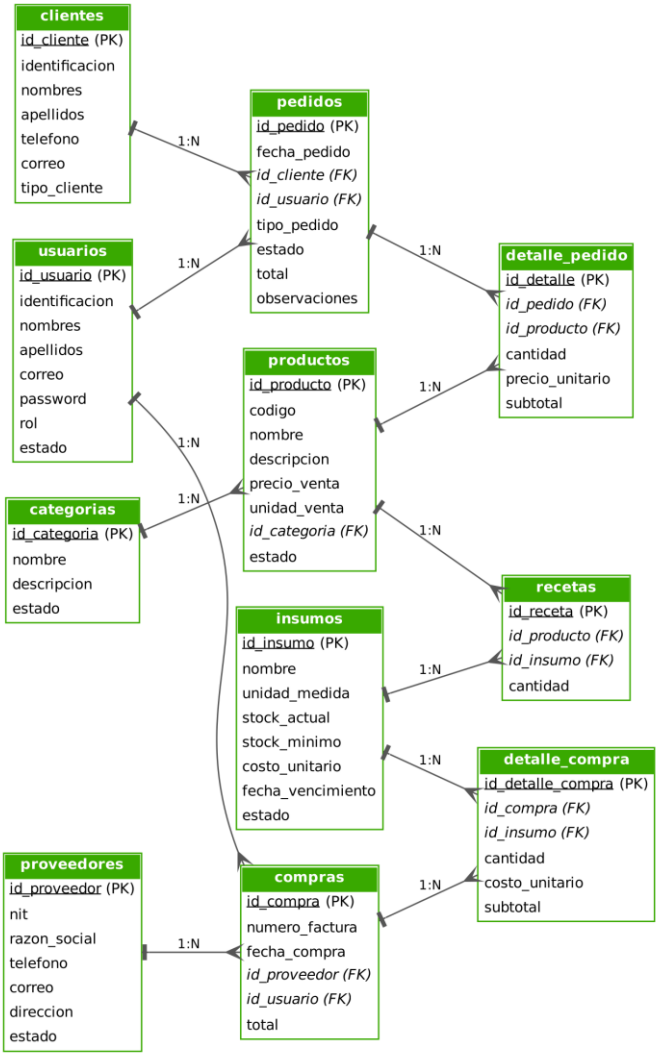


Figura 1. Propuesta inicial del modelo entidad–relación (debe ser validada y ajustada por el aprendiz).

Endpoints sugeridos

Los aprendices deberán diseñar los endpoints utilizando el prefijo `/api/v1`. Como punto de partida pueden considerar:

Autenticación

Método	Endpoint	Descripción
POST	/auth/login	Inicia sesión y retorna el token JWT (único endpoint público)
POST	/auth/registro	Registra un usuario (solo ADMIN)
GET	/auth/perfil	Datos del usuario autenticado

CRUD básico. Cada uno de los siguientes recursos debe implementar los cinco servicios: `GET /recurso`, `GET /recurso/{id}`, `POST /recurso`, `PUT /recurso/{id}` y `DELETE /recurso/{id}`.

Recurso	Ruta base	Roles permitidos
Usuarios	`/usuarios`	ADMIN
Categorías	`/categorias`	ADMIN (consulta: todos)
Productos	`/productos`	ADMIN (consulta: todos)
Insumos	`/insumos`	ADMIN, COCINERO
Recetas	`/recetas`	ADMIN, COCINERO
Proveedores	`/proveedores`	ADMIN
Compras	`/compras`	ADMIN
Clientes	`/clientes`	ADMIN, CAJERO
Pedidos	`/pedidos`	ADMIN, CAJERO (cambio de estado: COCINERO)
Calificaciones (MongoDB)	`/calificaciones`	ADMIN, CAJERO

Ejemplo — Productos

```
GET     /api/v1/productos
GET     /api/v1/productos/{id}
POST    /api/v1/productos
PUT     /api/v1/productos/{id}
DELETE  /api/v1/productos/{id}
```

Endpoints de consulta y procesos de negocio

Método	Endpoint	Descripción
`GET`	`/productos/categoria/{id_categoria}`	Productos de una categoría
`GET`	`/productos/{id}/receta`	Insumos y cantidades de la receta de un producto
`GET`	`/productos/{id}/costo`	Costo de producción (Σ cantidad × costo_unitario) y margen frente al precio de venta
`GET`	`/insumos/stock-bajo`	Insumos con stock_actual <= stock_minimo
`GET`	`/insumos/por-vencer?dias=7`	Insumos que vencen en los próximos N días
`GET`	`/proveedores/{id}/compras`	Historial de compras a un proveedor
`GET`	`/clientes/{id}/pedidos`	Historial de pedidos de un cliente
`GET`	`/pedidos/estado/{estado}`	Pedidos filtrados por estado
`PATCH`	`/pedidos/{id}/estado`	Cambia el estado del pedido (aplica RN03 y RN05)
`GET`	`/reportes/ventas?desde=&hasta=`	Total vendido y número de pedidos en un rango de fechas
`GET`	`/reportes/productos-mas-vendidos?limite=10`	Ranking de productos más vendidos
`GET`	`/calificaciones/producto/{id_producto}`	Calificaciones y promedio de un producto (MongoDB)

Arquitectura y estructura del proyecto

La API se construirá con **FastAPI** y siguiendo una arquitectura por capas: rutas (controladores), servicios (lógica de negocio), repositorios (acceso a datos), modelos y esquemas de validación. Estructura mínima sugerida:

```
api_gastronomia/
├── app/
│   ├── main.py              # punto de entrada
│   ├── core/                # config.py, security.py (JWT,
bcrypt)
│   ├── database/            # conexion_mysql.py,
conexion_mongo.py
│   ├── models/              # modelos de las tablas
│   ├── schemas/             # validaciones (Pydantic)
│   ├── repositories/        # consultas a la base de datos
│   ├── services/            # reglas de negocio (RN01..RN06)
│   └── routers/             # endpoints por recurso
├── database/
│   ├── 01_ddl_bd_gastronomia.sql
│   ├── 02_dml_datos_prueba.sql
│   ├── 03_dcl_usuarios.sql
│   └── 04_mongo_calificaciones.js
├── docker-compose.yml       # MySQL y MongoDB
├── requirements.txt
├── .env.example            # sin credenciales reales
└── README.md
```

Librerías sugeridas: `fastapi`, `uvicorn`, `sqlalchemy` o `pymysql`, `pymongo`, `pydantic`, `python-jose` o `PyJWT`, `passlib[bcrypt]` y `python-dotenv`.

Seguridad

Debido a que el sistema maneja información de clientes, ventas y costos, los aprendices deberán implementar mecanismos de seguridad con **JWT**, es decir, que todos los endpoints deben ser privados excepto `/auth/login`. Además:

- ✓ El token se envía en el encabezado `Authorization: Bearer <token>` y debe tener tiempo de expiración.
- ✓ Los permisos se validan por rol; la API responde **401** si no hay token o es inválido y **403** si el rol no tiene permiso.
- ✓ Las credenciales de bases de datos y la clave secreta del JWT se manejan en variables de entorno (`.env`), que no se suben a GitHub.
- ✓ La API se conecta a MySQL con un usuario propio de privilegios mínimos (creado con comandos DCL), no con root.

- ✓ Se validan los datos de entrada y se manejan los errores con códigos HTTP adecuados (400, 404, 409, 422, 500).

Documentación de la API

Cada endpoint debe estar documentado (descripción, parámetros, cuerpo de la petición, respuestas y códigos de estado) en Swagger/OpenAPI (`/docs`) y se debe entregar la colección de Postman exportada en el repositorio, con pruebas de cada servicio.

Nota: Todos los archivos, variables, clases, funciones, rutas y controladores deben tener en el nombre la letra inicial del nombre del aprendiz (ejemplo para Wilson: `w_productos_router.py`, `w_obtener_productos()`).

ANEXO: Documentación de la base de datos

Como anexo de esta actividad, el aprendiz deberá documentar la base de datos del proyecto diligenciando el formato **6. Manual de Configuración de Base de Datos**. El anexo debe contener las siguientes secciones del formato:

Sección del formato	Contenido esperado para la Unidad de Gastronomía
Historial de revisión y cambios	Versión, fechas y responsables de elaboración, revisión (instructor) y aprobación (líder de la unidad productiva).
Introducción	Propósito de la base de datos y su relación con la API REST de la Unidad Productiva de Gastronomía.
Alcance	Procesos cubiertos (recetas, inventario, compras, pedidos, ventas y calificaciones) y lo que queda fuera del prototipo.
Responsables e involucrados	Aprendiz (responsable), instructor (revisor) y líder de la unidad productiva (involucrado), con su rol.
Modelo Entidad Relación (MER)	Diagrama completo con entidades, atributos, llaves y cardinalidades, elaborado en MySQL Workbench, draw.io o similar.
Diccionario de datos	Por cada tabla: campo, tipo de dato, tamaño, nulo, llave (PK/FK/UQ), valor por defecto, restricción (CHECK) y descripción. Incluir la estructura de la colección de MongoDB.
Modelo relacional	Modelo físico generado desde MySQL Workbench (ingeniería inversa) mostrando tablas, tipos y relaciones.
Justificación del motor seleccionado	Comparativa de MySQL frente a PostgreSQL, SQL Server y Oracle, y justificación del uso de MongoDB para las calificaciones.
Requisitos de configuración	Herramientas y versiones: Docker, imágenes de MySQL y MongoDB, clientes (Workbench, Compass/Studio 3T), Python y librerías.
Scripts	DDL (base de datos, tablas, llaves, índices y restricciones), DML (mínimo 10 registros de prueba

	por tabla principal), DCL (usuario de la API con privilegios mínimos) y script de MongoDB con validación ($jsonSchema).
Configuración y ejecución de la base de datos	Paso a paso: levantar los contenedores, ejecutar los scripts y evidenciar la creación (SHOW TABLES, DESCRIBE, consultas) con capturas que muestren fecha y hora.
Otras consideraciones	Charset utf8mb4, zona horaria America/Bogota, respaldo y restauración (mysqldump / mongodump) y manejo seguro de credenciales.

Ejemplo de diccionario de datos (tabla `productos`):

Campo	Tipo	Nulo	Llave	Defecto / Restricción	Descripción
`id_producto`	INT	No	PK	AUTO_INCREMENT	Identificador del producto
`codigo`	VARCHAR(20)	No	UQ		Código interno
`nombre`	VARCHAR(100)	No			Nombre del producto
`precio_venta`	DECIMAL(10,2)	No		CHECK (precio_venta > 0)	Precio de venta
`id_categoria`	INT	No	FK	categorias(id_categoria)	Categoría del producto
`estado`	ENUM	No		'DISPONIBLE'	DISPONIBLE, AGOTADO, INACTIVO

Forma de Entrega:

Sustentar al instructor e incluir el enlace al repositorio de GitHub donde se encuentre disponible el código fuente de la solución, los scripts de la base de datos (carpeta `database/`) y la colección de Postman.
El anexo de documentación de la base de datos (formato Manual de Configuración de Base de Datos) se entrega como un archivo independiente y se debe subir al enlace suministrado por el instructor.

Fecha de Inicio:	14/10/2026
Fecha de Fin:	28/10/2026
Intensidad /duración	20 horas